

# Reviewer Pack — Coxeter Group Rank and Monomial Ideal Complexity

Entry 6562dd8c1790 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 11:05:36 UTC

## Coxeter Group Rank and Monomial Ideal Complexity

**Entry ID:** 6562dd8c1790 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 11:05:36 UTC

### 1. Conjecture

**Field A** (mathematical branch): Coxeter Theory **Field B** (complexity object): Complexity Theory: Monomial Ideal Complexity

#### Statement:

The rank of the Coxeter group associated with the monomial ideal corresponding to a given boolean function is upper bounded by the degree of the function, and this bound holds for all instances with at most  $n$  variables.

#### Rationale (proposer's reasoning):

Coxeter groups are known to encode combinatorial structures that can be used in counting problems. Monomial ideals, on the other hand, provide a combinatorial representation of boolean functions. The conjecture aims to link these two areas by proposing a quantitative relation between the rank of the Coxeter group and the complexity of the monomial ideal, which could expose structural properties relevant to complexity theory.

**Taxonomy category:** GCT\_DET\_PERM (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** d8226b84ef808406

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if, for all boolean functions with at most  $n$  variables and corresponding monomial ideals, the rank of the associated Coxeter group does not exceed the degree of the function, with a statistical mean rank across 30 random seeds being less than or equal to the degree.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - (Coxeter Group AND rank) AND (monomial ideal OR complexity theory)  
- (boolean function AND Coxeter group) AND monomial ideal complexity - degree of boolean function AND upper bound AND Coxeter group rank

**Top relevant hits considered:** - [http://arxiv.org/abs/hep-th/0610278v4] A Special Class of Rank 10 and 11 Coxeter Groups - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2 - [http://arxiv.org/abs/2303.14697v2] The central tree property and algorithmic problems on subgroups of free groups - [http://arxiv.org/abs/2505.08722v2] Properties of LCM Lattices of Monomial Ideals - [http://arxiv.org/abs/2406.18700v4] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [http://arxiv.org/abs/2409.11697v3] Monomial Matrix Group Equivariant Neural Functional Networks - [http://arxiv.org/abs/1009.4918v2] Bounding reflection length in an affine Coxeter group - [http://arxiv.org/abs/1107.1995v1] Lusztig's  $a$  function for Coxeter groups of rank 3

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=-9, elapsed=221.4s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations, permutations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def monomial_ideal(boolean_func):
        n = len(boolean_func)
        ideal = set()
        for i in range(1, n + 1):
            for comb in combinations(range(n), i):
                monomial = [1 if j in comb else 0 for j in range(n)]
                ideal.add(tuple(monomial))
        return ideal

    def coxeter_group_rank(ideal):
        n = len(next(iter(ideal)))
        generators = []
        for i in range(n):
```

```

        generators.append((1,) * i + (0,) * (n - i) + (-1,))
    relations = set()
    for gen1, gen2 in combinations(generators, 2):
        product = tuple(a * b for a, b in zip(gen1, gen2))
        if all(x != y for x, y in zip(product, (1,) * n)):
            relations.add((gen1, gen2))
    return len(relations)

def degree(boolean_func):
    n = len(boolean_func)
    max_length = 0
    for i in range(1 << n):
        binary_rep = bin(i)[2:].zfill(n)
        if all(binary_rep[j] == str(boolean_func[j]) for j in range(n)):
            length = sum(int(bit) for bit in binary_rep)
            if length > max_length:
                max_length = length
    return max_length

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    boolean_func = generate_boolean_function(n)
    ideal = monomial_ideal(boolean_func)
    rank = coxeter_group_rank(ideal)
    degree_value = degree(boolean_func)
    results.append((n, rank, degree_value))

mean_rank = sum(rank for _, rank, _ in results) / len(results)
support_fraction = all(rank <= degree_value for _, rank, degree_value in results)

return {
    "metric_name": "Coxeter Group Rank",
    "metric_value": mean_rank,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction,
    "counterexample": "" if support_fraction else "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    results = [run_trial(seed)["metric_value"] for seed in seeds]
    mean_result = sum(results) / len(results)
    support_fraction = all(result <= n for result, n in zip(results, [5, 10, 15, 20, 30, 40]))

    if support_fraction:
        print(f"RESULT: SUPPORTED mean={mean_result} std=0.0 support_fraction=1.0")
    elif any(result > n for result, n in zip(results, [5, 10, 15, 20, 30, 40])):
        first_failing_seed = seeds[results.index(max(results))]
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

```
else:
    print("RESULT: INCONCLUSIVE reason=unsupported_operation")
```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

(empty)

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed before producing any data, which means we cannot verify if the conjecture holds or not. | next: Re-run the test and ensure it completes without crashing to verify the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 15445        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12119        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 8960         |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8033         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9554         |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11112        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 6549         |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9872         |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9590         |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 74440        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 165674 ms total latency. Provider mix: {'ollama\_remote': 10}

(full prompt+response transcripts available in [research/audit/6562dd8c1790.jsonl](#))

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/6562dd8c1790.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/6562dd8c1790.tar.gz` (if generated)